



help me write a prompt so each time I feed perplexity the QA bug report it generates both tasks.md and SPRINT_QA_TASKS.md so we have this type of orchestration going

Here's the meta-prompt you paste into Perplexity (or any AI) each time you have a new QA bug report PDF:

```
You are a senior software architect converting a QA bug report into two structured t
```

```
## Your inputs
```

- The QA bug report attached to this message (PDF or pasted text)
- The repository structure at: github.com/cfroszte/softomedia-live2026 (main branch)

```
## Your outputs
```

```
You must produce EXACTLY two files. Output them in full, one after the other.
```

```
---
```

```
### FILE 1: `tasks.md`
```

```
A comprehensive developer specification. For each bug in the report, write a task e
```

1. **Task header** – task number, bug title, route/URL, priority (High/Medium/Low),
2. **What to do** – 2-4 sentences explaining the bug and the fix goal in plain English
3. **Files to edit** – list each file with:
 - Full repo path (e.g., `client-app/src/pages/admin/UserManagement.jsx`)
 - Bullet-point instructions scoped to that file only
 - Any inline code snippet needed (helper functions, Zustand store shape, API call)
4. **Data mapping table** – where applicable, a markdown table mapping UI label → Fi
5. **Cascade warning** – if deleting/updating a record affects child collections, ca
6. **Implementation notes** – gotchas, soft-delete vs hard-delete decisions, role gu

```
At the bottom of `tasks.md` include:
```

- A **"Do Not Touch"** section listing any confirmed-working areas from the QA repor
- A **Reference Files table** listing: `docs/database\_schema.md`, `docs/data_diction
- A **Notes for Developer** section covering: icon library (lucide-react), state mar

```
---
```

```
### FILE 2: `SPRINT_QA_TASKS.md`
```

```
An AI-execution breakdown optimised for low-context-window models (OpenRouter free
```

1. Each task in `tasks.md` must be broken into **atomic sub-tasks** – one sub-task =

```

2. Numbering format: `TASK <sprint>.<subtask>` (e.g., TASK 1.1, TASK 1.2). Group su
3. Each sub-task entry must contain ONLY:
  - `### TASK X.Y – <short title>`
  - `– **File:** <exact repo path>`
  - `– **Action:** <one-sentence description of what to add/change/fix>`
  - `– **Logic:** <bullet list of the exact implementation steps for that one file -`
4. Sub-task ordering rule: backend repository methods first, then backend API route
5. At the bottom, include:
  - A “Do Not Modify” table (confirmed-working areas from the QA report)
  - A File Reference Map table: Bug Area | Primary Frontend File | Primary Back

---

## Orchestration contract between the two files
- Every bug in `tasks.md` must have at least one corresponding sub-task in `SPRINT_QA_TASKS.md`
- The sub-task in `SPRINT_QA_TASKS.md` must reference the exact same file paths as `tasks.md`
- `SPRINT_QA_TASKS.md` contains NO inline code – all code lives in `tasks.md`
- The Roo Code agent will read `SPRINT_QA_TASKS.md` to decide WHAT file to open and

---

## Output format rules
- Output FILE 1 (`tasks.md`) in full first, inside a markdown code block labelled `tasks.md`
- Then output FILE 2 (`SPRINT_QA_TASKS.md`) in full, inside a markdown code block labelled `SPRINT_QA_TASKS.md`
- Do not summarise, truncate, or use placeholder text like "...add remaining tasks"
- Do not invent file paths – derive them from the repository structure. If a file path is missing, use the QA report
- Preserve the exact Firestore field naming convention from the QA report (camelCase)

## Stack conventions to apply automatically
- Icons: `lucide-react` only
- State: Zustand for cross-component; local React state for single-component (forms,
- Soft-delete: `status = 'inactive'` for retailers/stores; `status = 'suspended'` for
- API errors: every POST/PUT/PATCH/DELETE must have a visible success and error UI response
- Lazy loading: never load all stores/screens on mount – load only what the current screen needs
- Role guards: Super Admin-only features must check `user.role === 'superadmin'` before rendering

Now process the attached QA bug report and produce both files in full.

```

How to use this going forward

1. Open a new Perplexity thread (or any AI with file upload)
2. Attach your new QA PDF
3. Paste this prompt above it
4. Get back both complete files
5. Commit both to main in your repo
6. Paste the Roo Code orchestration prompt (the one from earlier) and it picks up from Task 1.1 automatically

The key design is the **orchestration contract** paragraph — it locks both files into a dependency relationship so neither can drift out of sync with the other.

